

```
1  
2  
3  'TypeScript' {  
4  
5      [ Workshop ]  
6  
7  
8  
9      < For Beginners >  
10  
11  
12  }  
13  
14
```



Aashish Thapa
voidash



Mahip Adhikari
themahip

Contents

- Why TypeScript
 - What is static typing
- Case Study
- Follow along: Lets get coding
- Data Types
- Classes
- Interface { Union & Intersection }
- Guards & Generics
- Decorators
- Installation & Tooling { popular IDE }

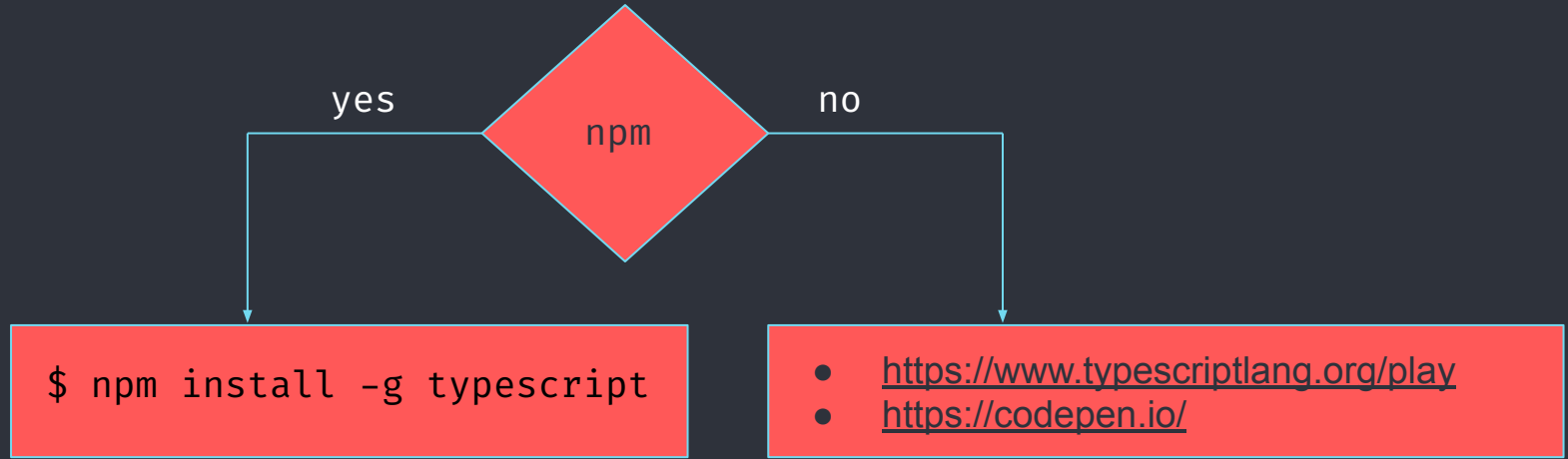
Clearing the Basics

- Compiler & Interpreter
- Compile Time & Runtime
- Type System
 - Static and Dynamic Typing
- Type Inference
- Transpiling vs Compiling
- Coercion

Javascript vs Typescript

- Typescript is superset of Javascript
- Javascript is dynamically typed whereas Typescript is statically typed
- Typescript can be compiled to any version of javascript so we can use new features and maintain backwards compatibility
- Typescript code is more predictable

Follow Along



01 {

[The Problem with JS]

< Type Inference, Hunting
Bugs, IDE support >

}

I have a new framework
down here



JavaScript Developers



JS



JS

$x \% 2 !== 0$

is-odd npm: v5.6.3 Downloads: 2,361,000 Downloads: 2.3M Downloads: 2.3M Downloads: 2.3M

Returns true if the given number is odd, and is an integer that does not exceed the JavaScript MAXIMUM_SAFE_INTEGER.

Please consider following this project's author, [Jen Schlinkert](#), and consider starring the project to show your ❤️ and support.

Install

Install with npm:

\$ npm install --save is-odd

Usage

Works with strings or numbers.

```
const isOdd = require('is-odd');  
  
console.log(isOdd('1')); // => true  
console.log(isOdd('9')); // => true
```

Weekly Downloads
269,847

downloads 2.3M/month

downloads 59M

1 <https://wtfjs.com>

2
3
4 **"b" + "a" + +"a" + "a";**

5 **parseInt("f*ck");**

6 **parseInt("f*ck", 16);**

7 **true + true;**

8 **(true + true) * (true + true) - true;**

9
10
11

12

13

14

1 <https://wtfjs.com>

2
3 `"b" + "a" + +"a" + "a"; // → 'baNaNNa'`

4
5 `parseInt("f*ck"); // → NaN`

6
7 `parseInt("f*ck", 16); // → 15`

8
9 `true + true; // → 2`

10
11 `(true + true) * (true + true) - true; // → 3`

12 JS is inherently flawed in some aspect. Typescript being
13 superset can help but can't solve it...
14

Find The Errors; {

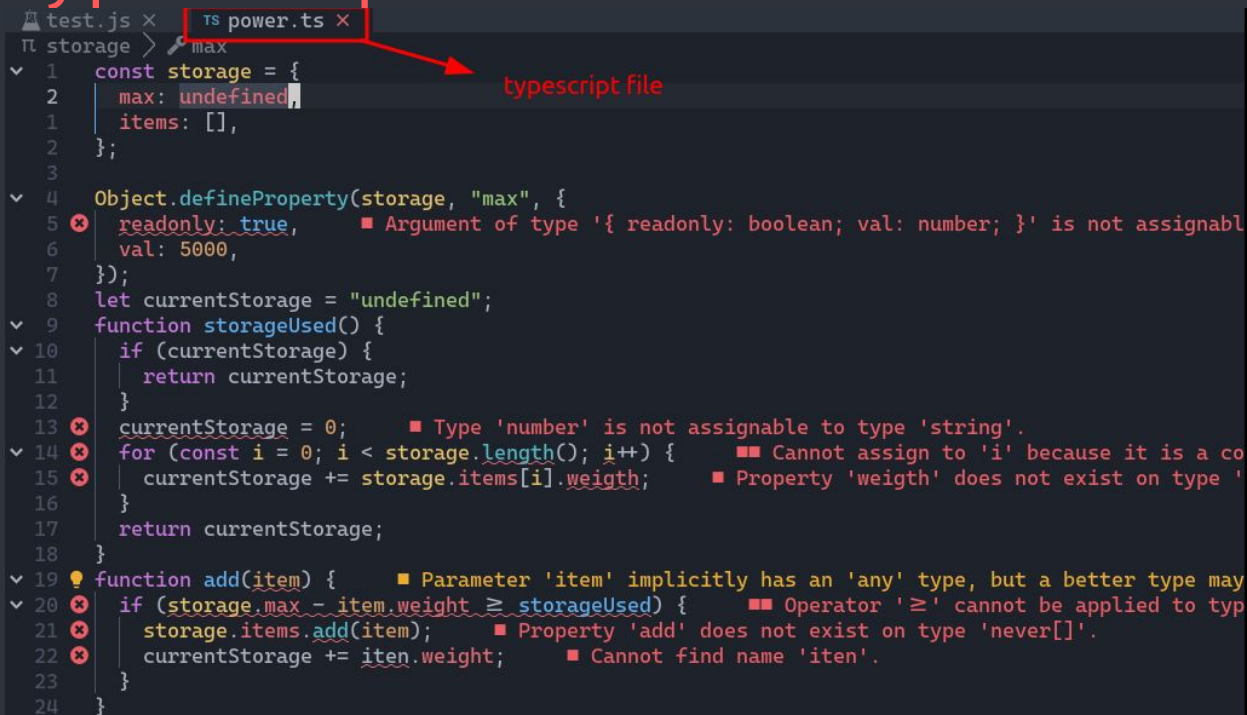
```
1  const storage = {  
2    max: undefined,  
3    items: [],  
4  };  
5  
6  Object.defineProperty(storage, "max",  
7    {  
8      readonly: true,  
9      val: 5000,  
10     });  
11  
12  let currentStorage = "undefined";  
13  function storageUsed() {  
14    if (currentStorage) {  
      return currentStorage;  
    }  
    currentStorage = 0;  
    for (const i = 0; i <  
      storage.length(); i++) {  
        currentStorage +=  
        storage.items[i].weight;  
      }  
    return currentStorage;  
  }
```

```
function add(item) {  
  if (storage.max - item.weight >= storageUsed) {  
    storage.items.add(item);  
    currentStorage += item.weight;  
  }  
}  
add({ weight: 3000 });  
console.log("hello");
```

JS Tooling

```
test.js x  ts power.ts x
storage >  constructor >  max
1  const storage = {
2    max: undefined,
   items: [],
   };
3
4  Object.defineProperty(storage, "max", {
5    readonly: true,
6    val: 5000,
7  });
8  let currentStorage = "undefined";
9  function storageUsed() {
10   if (currentStorage) {
11     return currentStorage;
12   }
13   currentStorage = 0;
14   for (const i = 0; i < storage.length(); i++) {
15     currentStorage += storage.items[i].weight;
16   }
17   return currentStorage;
18 }
19 function add(item) {
20   if (storage.max - item.weight ≥ storageUsed) {
21     storage.items.add(item);
22     currentStorage += item.weight;
23   }
24 }
```

TypeScript to Rescue



```
test.js x ts power.ts x
storage > max
1 const storage = {
2   max: undefined,
3   items: [],
4 };
5
6 Object.defineProperty(storage, "max", {
7   readonly: true,
8   val: 5000,
9 });
10 let currentStorage = "undefined";
11 function storageUsed() {
12   if (currentStorage) {
13     return currentStorage;
14   }
15   currentStorage = 0;
16   for (const i = 0; i < storage.length(); i++) {
17     currentStorage += storage.items[i].weigh;
18   }
19   return currentStorage;
20 }
21 function add(item) {
22   if (storage.max - item.weight ≥ storageUsed) {
23     storage.items.add(item);
24     currentStorage += iten.weight;
25   }
26 }
```

typescript file

```
1 02 {
```

```
2  
3  
4  
5 [Types]
```

```
6  
7  
8 < Number, String, Enum,  
9 Void, Any >
```

```
10  
11  
12 }  
13  
14
```

What are Types {

< A type is a classification of data that defines the operations that can be done on that data, the meaning of the data, and the set of allowed values. Typing is checked by the compiler and/or run time to ensure the integrity of the data, enforce access restrictions, and interpret the data as meant by the developer.>

Different Types



Number



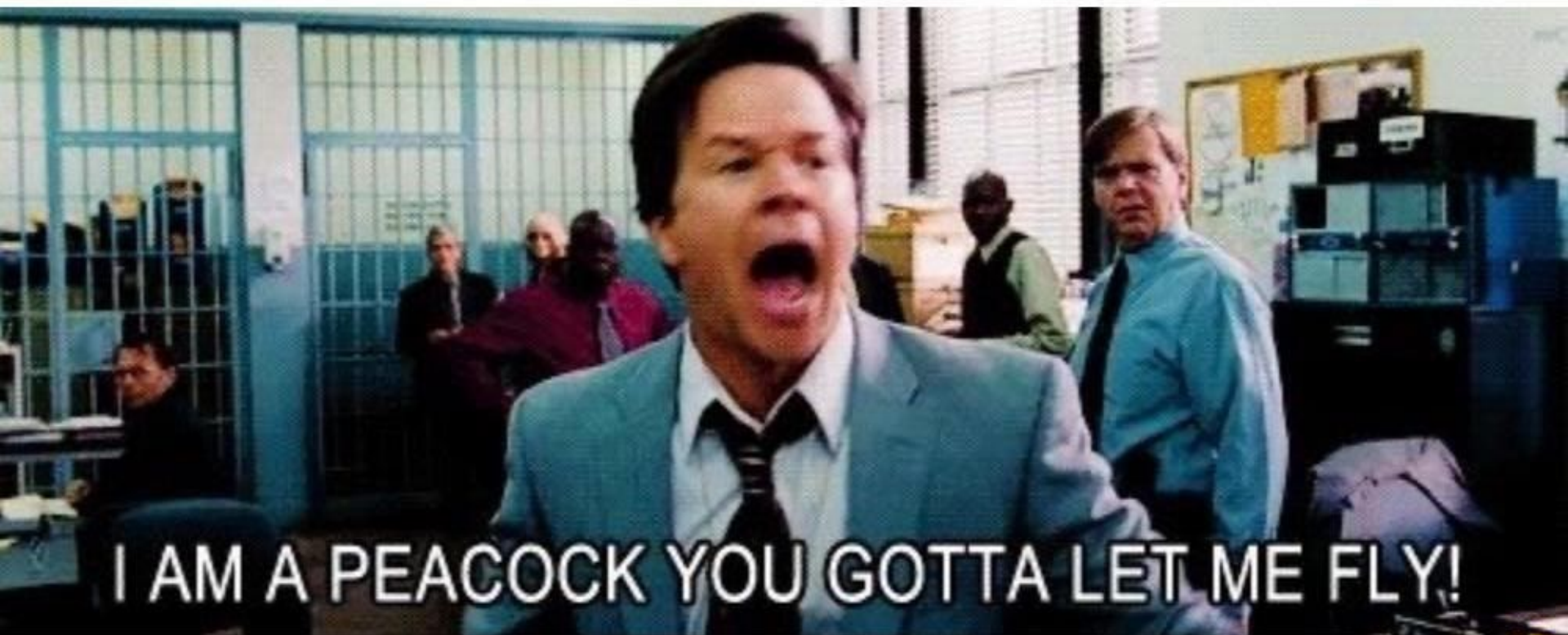
String



Boolean

}

JavaScript devs who migrate to TypeScript
and can't get past compiler errors



I AM A PEACOCK YOU GOTTA LET ME FLY!

Concepts < /1 > {



Type Inference? Weak
Typing & Strong Typing

}

```
const x = 10; //inferred as "number"  
const y = "hello"; //inferred as "string"  
const z = x + y;  
console.log(z); // Output: 10hello
```

Concepts < /2 > {



Syntax vs Semantics

}

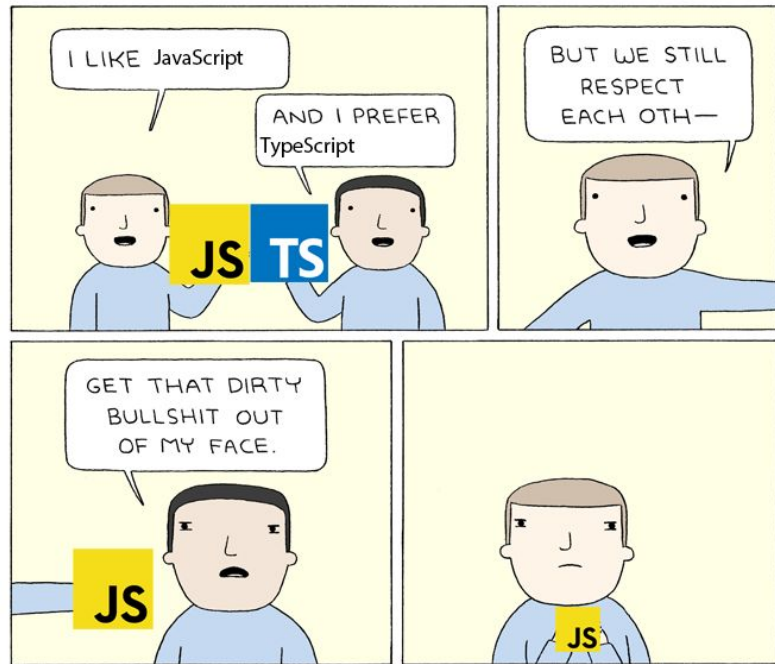
```
const x = "5";  
const y = 10;  
const result = x + y;  
console.log(result); // Output: "510"
```


Example

```
1 // Boolean
2 let isDone: boolean = false;
3
4 // Number
5 let decimal: number = 6;
6 let hex: number = 0xdeadbeef;
7 let binary: number = 0b1101;
8 let octal: number = 0o744;
9
10 // String
11 let color: string = "blue";
12 let fullName: string = `jamyang
13 Gelek`;
14 let sentence: string = `Hello, my
   name is ${fullName}.`;
15
16 // Array
17 let list: number[] = [1, 2, 3];
18 let anotherList: Array<number> = [4,
   5, 6];
```

```
// Tuple
let tuple: [string, number] = ["hello", 10];
19
20 // Enum
21 enum Color {
22     Red,
23     Green,
24     Blue,
25 }
26 let c: Color = Color.Green;
27
28 // Any
29 let notSure: any = 4;
30 notSure = "khai ta";
31 notSure = false;
```

When you don't know
what type you should
use in TypeScript



Functions Type

- Function with no return value

```
function logMessage(message: string | number): void {  
    console.log(message);  
}
```

- Function with specific return value

```
function addNumbers(a: number, b: number): number {  
    return a + b;  
}
```

- Optional and Default Parameters

```
function sayHello(name?: string, origin="Nepal") {  
    console.log(`Hello, ${name ?? 'World'} from ${origin}!`);  
}
```

You can add Types with JSDoc

<https://jsdoc.app/>

```
/**
 * Adding two numbers. This annotation tells
 * TypeScript
 * which types to expect. Two parameters (params) of
 * type number and a return type of number
 *
 * @param {number} numberOne
 * @param {number} numberTwo
 * @returns {number}
 */
function addNumbers(numberOne, numberTwo) {
  return numberOne + numberTwo
}

console.log(addNumbers(3,4,5));
```

Type Aliasing

a type alias is a way to create a new name for an existing type. It allows you to give a type a more descriptive or meaningful name, making your code more readable and maintainable.

```
type UserId = string | number;
function getUserById(id: UserId): { id: UserId, name: string } {
  // code to fetch user by ID
}
const user1 = getUserById('abc123'); // id: string, name: string
const user2 = getUserById(123); // id: number, name: string
```

Custom Types

- Placed on `.d.ts` files

```
type Assignment = {  
  id: number,  
  weight: number,  
  filePath: String  
}  
  
type AssignmentStorage = {  
  id: number,  
  items: Assignment[]  
}
```

Type Assertions

You know more about the type that the typescript so the compiler shouldn't second guess you

- To override typescript checking

```
// Getting a reference to a DOM element with the ID "my-element"
```

```
const myElement = document.getElementById('my-element');
```

```
// Using a type assertion to tell TypeScript that myElement is an  
HTMLInputElement
```

```
const myInput = myElement as HTMLInputElement;
```

03 {

[Working With Classes]

< Constructors, Methods
etc>

}

INHERITANCE

Mera baccha hai tu

ABSTRACTION

Mai Nahi Dikhaunga

ENCAPSULATION

Jhund Bana Ke Chalo....Jhund Bana Ke Chalo

POLYMORPHISM

@crazy_shikhu

Classes

```
1
2
3  class Greeter {
4      greeting: string;
5
6      constructor(message: string) {
7          this.greeting = message;
8      }
9
10     greet() {
11         return "Hello, " + this.greeting;
12     }
13
14     let greeter = new Greeter("world");
```

Inheritance

```
class Animal {  
  move(distanceInMeters: number = 0) {  
    console.log(`Animal moved ${distanceInMeters}m.`);  
  }  
}  
  
class Dog extends Animal {  
  bark() {  
    console.log("Woof! Woof!");  
  }  
}  
  
const dog = new Dog();  
dog.bark();  
dog.move(10);  
dog.bark();
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14



Public, Private and Protected Modifiers

- Anybody can use it: Public
- Only the class itself and its method can use it: Private
- Those within the family can use it: Protected

```
class Animal {  
  private name: string;  
  protected age: number;  
  public breed: string;  
  constructor(name: string, age: number,  
    breed: string) {  
    this.name = name;  
    this.age = age;  
    this.breed = breed;  
  }  
}
```

```
public getName(): string {  
  return this.name;  
}  
protected getAge(): number {  
  return this.age;  
}  
}  
class Dog extends Animal {  
  public bark(): void {  
    console.log("Woof!");  
  }  
}  
const dog = new Dog("mithi", 5, "sheperd");
```

04 {

[Interfaces]

<Union, Intersection and
stuff>

}

Interfaces

- Defines the shape that values have
- It can be extended.

```
type Client = {  
  name: string;  
};  
  
interface VIPClient extends Client {  
  benefits: string[]  
}
```

- If two interfaces share same name then they can be merged.

<pre>interface Client { name: string; } interface Client { age: number; }</pre>	<pre>const harry: Client = { name: 'Ram', age: 41 }</pre>
--	---

Type Theory



- In type theory intersection, union and product have different meaning. Don't get confused...

Not your Math class

Union

```
1  // Union type example
2
3  interface Dog {
4      name: string;
5      breed: string;
6  }
7  interface Cat {
8      name: string;
9      color: string;
10 }
11
12 type Pet = Dog | Cat;
13
14 function printPet(pet: Pet) {
15     console.log(`Name: ${pet.name}`);
16     if ('breed' in pet) {
17         console.log(`Breed: ${pet.breed}`);
18     } else {
19         console.log(`Color: ${pet.color}`);
20     }
21 }
```

Intersection

```
1
2
3 type Admin = {
4     name: string;
5     privileges: string[];
6 };
7
8 type Employee = {
9     name: string;
10    startDate: Date;
11 };
12
13 type ElevatedEmployee = Admin & Employee;
14
15 let jackie : ElevatedEmployee = {
16     name: "Jackie",
17     privileges: ['management', 'admin panel'],
18     startDate: new Date().today()
19 }
```

05 {

[Type Guards]

<typeof, in, instanceof>

}



Type Guards

- For Primitive Literals

```
//use typeof
type Combinable = string | number;
function add(a: Combinable, b: Combinable) {
  // type guard
  if (typeof a === 'string' || typeof b === 'string') {
    return a.toString() + b.toString();
  }
}
```

- For Objects

```
type UnknownEmployee = Employee | Admin;
function printEmployeeInformation(emp: UnknownEmployee) {
  // type guard
  if ('privileges' in emp)
    console.log('Privileges'+ emp.privileges);
}
```

Type Guards

- For classes

```
//use instanceof
class Truck {
  drive() {
    console.log('Driving');
  }
  loadCargo(amount: number) {
    console.log('Loading cargo ...' + amount);
  }
}
class Car {
  drive() {
    console.log('Driving');
  }
}

type Vehicle = Car | Truck ;
const v1 = new Car();
const v2 = new Truck();
```

```
// use instanceof

function useVehicle(vehicle: Vehicle)
{
  vehicle.drive();
  // type guarding
  if (vehicle instanceof Truck) {
    vehicle.loadCargo(12);
  }
}
```


Tod

1

2

3

4

5

6

7

8

9

10

11

12

13

14

06 {

[Generics]

<interface, <T>, etc>

}

Generics

```
// defining a interface  
with generic data type
```

```
interface person<T>{  
    id:number,  
    name:string,  
    role:string;  
    data:T  
}
```

```
const Rohan:person<object>={  
    id:1,  
    name:"Rohan Sharma",  
    role:"User",  
    data:{  
        age:33,  
        pictureid:"00A12R8965"  
    }  
}
```

```
1
2      07 {
3
4
5      [Enum]
6
7
8      <stuff>
9
10
11
12     }
13
14
```

index.html

workshop.ts

1

2

3

4

5

6

7

8

9

10

11

12

13

14

Further Exploration

- Modules
- tsconfig.json
- Namespaces

Resources

Typescript Classes:

<https://www.typescriptlang.org>

Awesome Typescript

<https://github.com/dzharii/awesome-typescript>

Theme from Slidesgo

